

3DOD_snow visualization

- point cloud augmentation

이상욱

Why doesn't snow simulation affect Pedestrian
much?

- Point cloud removing percentage

Why doesn't snow simulation affect Pedestrian much?

- Point cloud removing percentage

```
===== [ Class-wise Point Density Analysis ] =====
[Car] (총 14305개 유효 객체)
- 원본 평균 포인트 수 : 499.93 pts
- 눈(Snow) 평균 포인트 수: 240.46 pts
- 평균 포인트 유실률 : 63.04 %

-----
[Pedestrian] (총 2279개 유효 객체)
- 원본 평균 포인트 수 : 209.89 pts
- 눈(Snow) 평균 포인트 수: 193.26 pts
- 평균 포인트 유실률 : 11.85 %

-----
[Cyclist] (총 892개 유효 객체)
- 원본 평균 포인트 수 : 164.58 pts
- 눈(Snow) 평균 포인트 수: 135.17 pts
- 평균 포인트 유실률 : 36.53 %

-----
```

- Raw와 snow 처리 후의 point cloud에 대해 GT bbox 범위 내의 유실률을 체크해봤습니다.
- 파란색 박스의 결과는 전체 유실률을 확인한 것이고, 오른쪽은 시각화했던 scene들에 대해 개별 point 유실률을 체크한 것입니다.
- 포인트 유실률이 평균 11.75%로 가장 낮다보니, prediction 에도 큰 영향을 미치지 않은 것으로 해석할 수 있을 것이라 생각했습니다.

```
===== [ Scene 000201 개별 객체 분석 ] =====
Obj 01 [Car] : 원본 2724개 -> 눈 적용 538개 | 유실률: 80.2%
Obj 02 [Car] : 원본 2315개 -> 눈 적용 850개 | 유실률: 63.3%
Obj 03 [Car] : 원본 639개 -> 눈 적용 307개 | 유실률: 52.0%
Obj 04 [Cyclist] : 원본 122개 -> 눈 적용 108개 | 유실률: 11.5%
Obj 05 [Car] : 원본 305개 -> 눈 적용 104개 | 유실률: 65.9%
Obj 06 [Car] : 원본 247개 -> 눈 적용 135개 | 유실률: 45.3%
Obj 07 [Car] : 원본 112개 -> 눈 적용 62개 | 유실률: 44.6%
Obj 08 [Car] : 원본 4개 -> 눈 적용 1개 | 유실률: 75.0%
Obj 09 [Car] : 원본 41개 -> 눈 적용 8개 | 유실률: 80.5%
Obj 10 [Pedestrian] : 원본 10개 -> 눈 적용 3개 | 유실률: 70.0%
Obj 11 [Pedestrian] : 원본 9개 -> 눈 적용 2개 | 유실률: 77.8%
Obj 12 [Cyclist] : 원본 81개 -> 눈 적용 67개 | 유실률: 17.3%

=====
```

```
===== [ Scene 006024 개별 객체 분석 ] =====
Obj 01 [Car] : 원본 6323개 -> 눈 적용 3288개 | 유실률: 48.0%
Obj 02 [Cyclist] : 원본 35개 -> 눈 적용 32개 | 유실률: 8.6%
Obj 03 [Pedestrian] : 원본 107개 -> 눈 적용 103개 | 유실률: 3.7%
Obj 04 [Pedestrian] : 원본 188개 -> 눈 적용 187개 | 유실률: 0.5%
Obj 05 [Car] : 원본 59개 -> 눈 적용 10개 | 유실률: 83.1%

=====
```

```
===== [ Scene 006496 개별 객체 분석 ] =====
Obj 01 [Car] : 원본 6271개 -> 눈 적용 3266개 | 유실률: 47.9%
Obj 02 [Pedestrian] : 원본 308개 -> 눈 적용 296개 | 유실률: 3.9%
Obj 03 [Pedestrian] : 원본 138개 -> 눈 적용 118개 | 유실률: 14.5%
Obj 04 [Pedestrian] : 원본 458개 -> 눈 적용 437개 | 유실률: 4.6%
Obj 05 [Cyclist] : 원본 36개 -> 눈 적용 31개 | 유실률: 13.9%
Obj 06 [Pedestrian] : 원본 153개 -> 눈 적용 148개 | 유실률: 3.3%
Obj 07 [Pedestrian] : 원본 35개 -> 눈 적용 29개 | 유실률: 17.1%
Obj 08 [Cyclist] : 원본 338개 -> 눈 적용 320개 | 유실률: 5.3%
Obj 09 [Car] : 원본 34개 -> 눈 적용 1개 | 유실률: 97.1%
Obj 10 [Car] : 원본 28개 -> 눈 적용 1개 | 유실률: 96.4%

=====
```

```
===== [ Scene 001221 개별 객체 분석 ] =====
Obj 01 [Car] : 원본 1841개 -> 눈 적용 965개 | 유실률: 47.6%
Obj 02 [Cyclist] : 원본 638개 -> 눈 적용 558개 | 유실률: 12.5%
Obj 03 [Car] : 원본 476개 -> 눈 적용 197개 | 유실률: 58.6%
Obj 04 [Car] : 원본 1018개 -> 눈 적용 739개 | 유실률: 27.4%
Obj 05 [Car] : 원본 274개 -> 눈 적용 102개 | 유실률: 62.8%
Obj 06 [Car] : 원본 207개 -> 눈 적용 130개 | 유실률: 37.2%
Obj 07 [Car] : 원본 82개 -> 눈 적용 26개 | 유실률: 68.3%
Obj 08 [Car] : 원본 56개 -> 눈 적용 28개 | 유실률: 50.0%
Obj 09 [Pedestrian] : 원본 12개 -> 눈 적용 8개 | 유실률: 33.3%
Obj 10 [Pedestrian] : 원본 21개 -> 눈 적용 11개 | 유실률: 47.6%
Obj 11 [Cyclist] : 원본 475개 -> 눈 적용 407개 | 유실률: 14.3%

=====
```

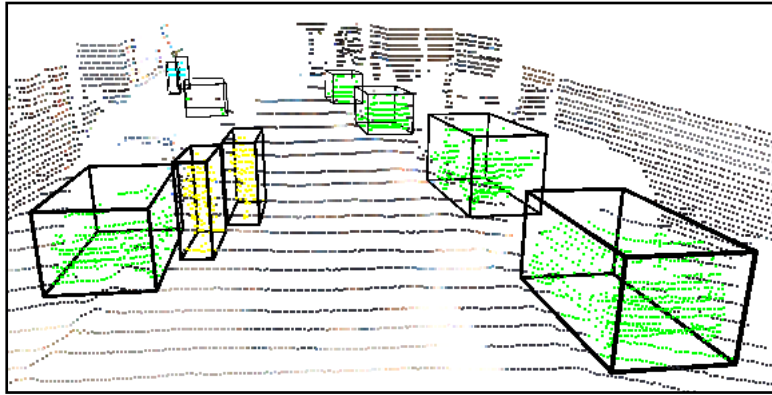
```
===== [ Scene 006813 개별 객체 분석 ] =====
Obj 01 [Car] : 원본 296개 -> 눈 적용 167개 | 유실률: 43.6%
Obj 02 [Cyclist] : 원본 104개 -> 눈 적용 85개 | 유실률: 18.3%
Obj 03 [Cyclist] : 원본 60개 -> 눈 적용 45개 | 유실률: 25.0%
Obj 04 [Pedestrian] : 원본 59개 -> 눈 적용 45개 | 유실률: 23.7%
Obj 05 [Cyclist] : 원본 40개 -> 눈 적용 12개 | 유실률: 70.0%
Obj 06 [Pedestrian] : 원본 51개 -> 눈 적용 40개 | 유실률: 21.6%
Obj 07 [Pedestrian] : 원본 26개 -> 눈 적용 22개 | 유실률: 15.4%
Obj 08 [Pedestrian] : 원본 17개 -> 눈 적용 14개 | 유실률: 17.6%
Obj 09 [Cyclist] : 원본 107개 -> 눈 적용 75개 | 유실률: 29.9%
Obj 10 [Pedestrian] : 원본 20개 -> 눈 적용 18개 | 유실률: 10.0%
Obj 11 [Pedestrian] : 원본 56개 -> 눈 적용 43개 | 유실률: 23.2%
Obj 12 [Pedestrian] : 원본 26개 -> 눈 적용 21개 | 유실률: 19.2%
Obj 13 [Car] : 원본 64개 -> 눈 적용 8개 | 유실률: 87.5%

=====
```

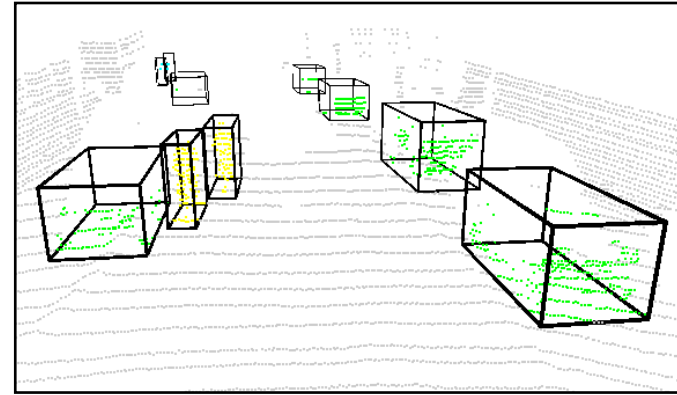
Why doesn't snow simulation affect Pedestrian much?

Scene - 000201

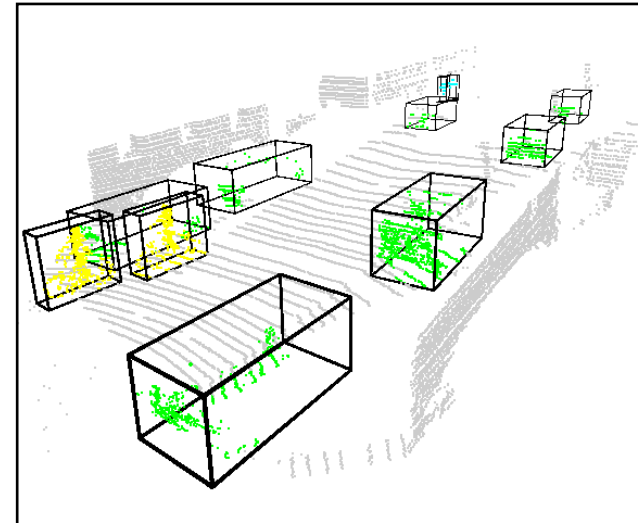
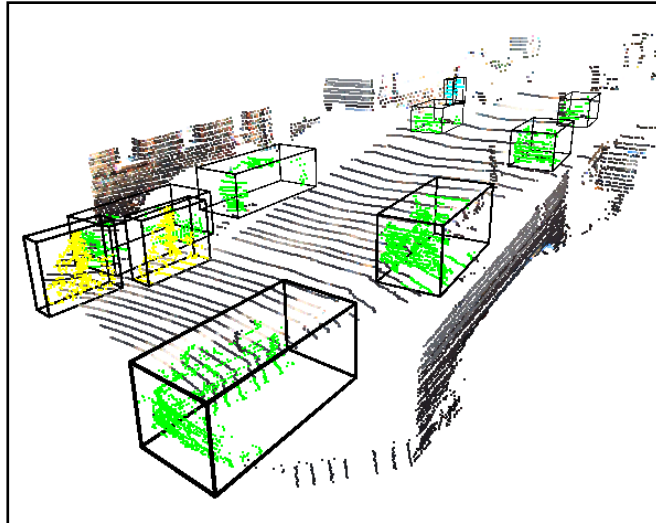
Raw kitti point cloud



Snow kitti point cloud



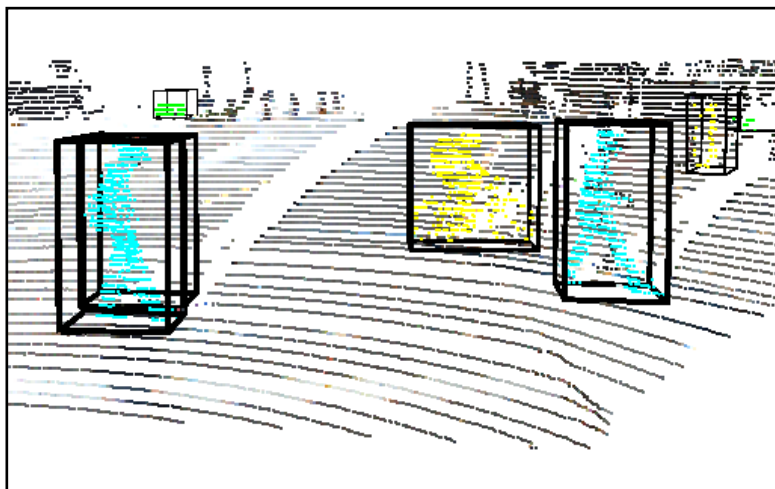
Scene - 001221



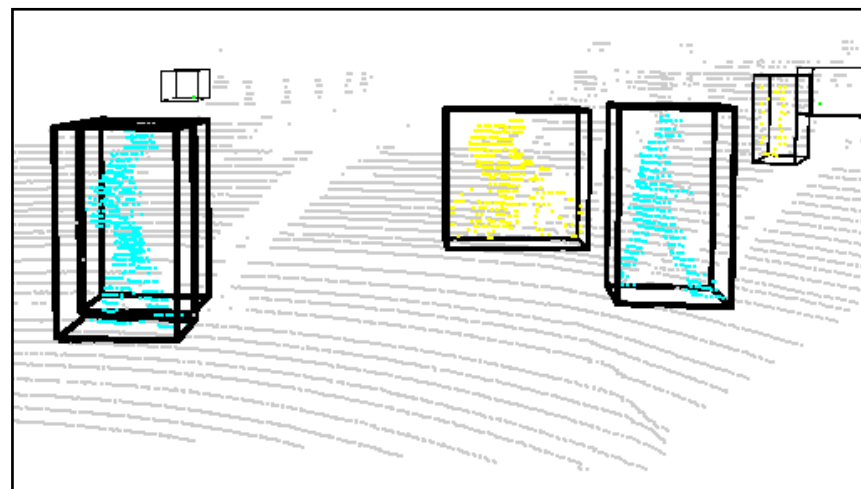
Why doesn't snow simulation affect Pedestrian much?

Scene - 006496

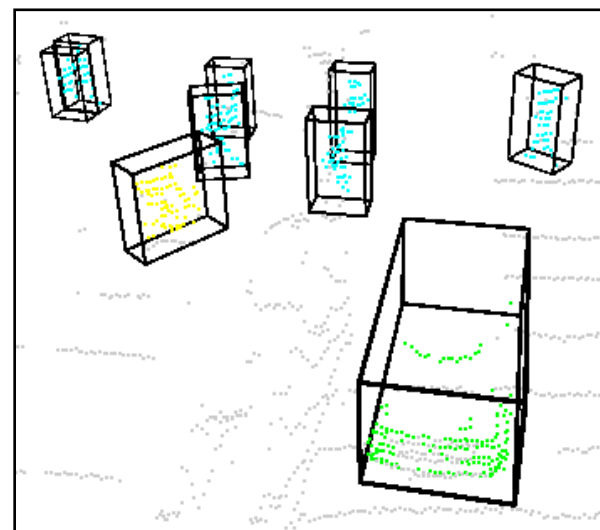
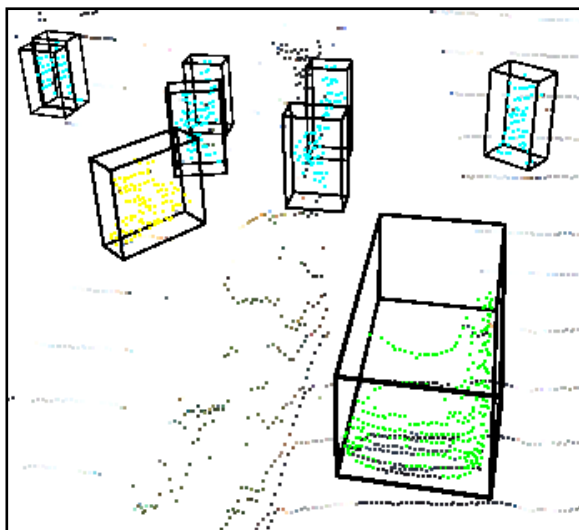
Raw kitti point cloud



Snow kitti point cloud

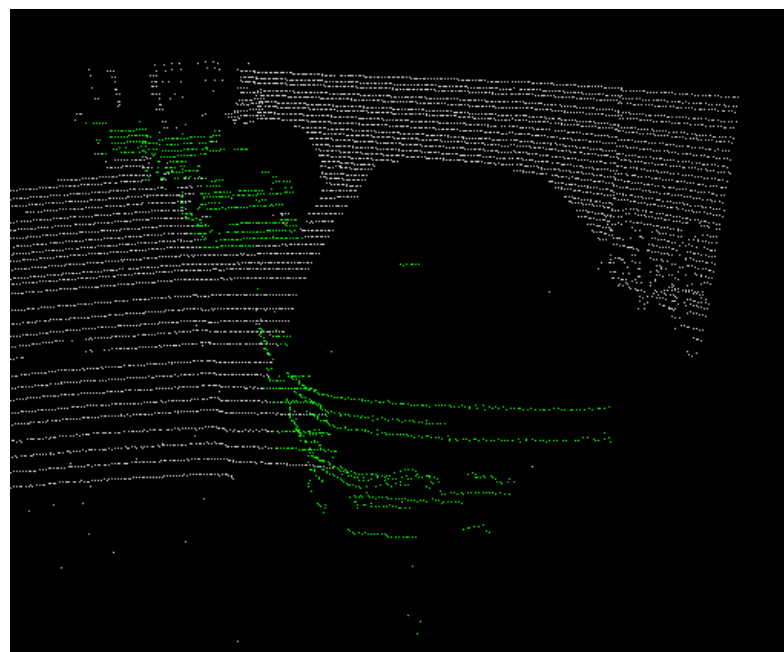


Scene - 006813

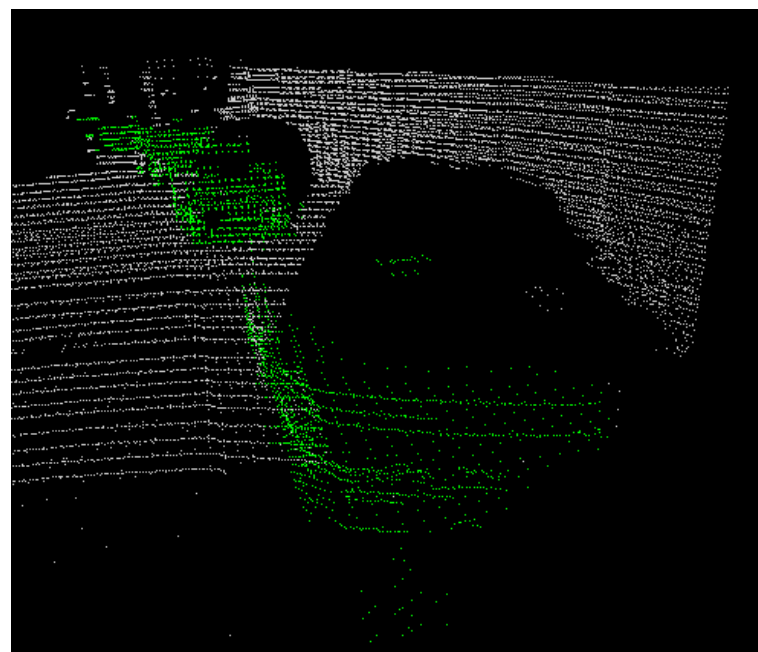


Object-aware point cloud augmentation
develop

	Car	Pedestrian	Cyclist
GT	초록색	하늘색(cyan)	노란색
Pred	빨간색	보라색(magenta)	주황색



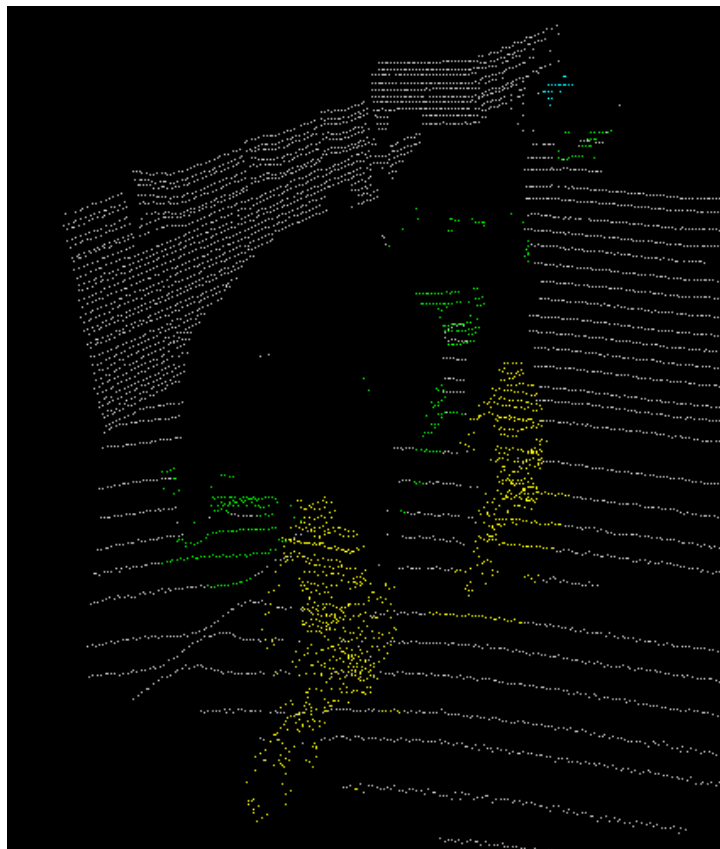
snow kitti point cloud



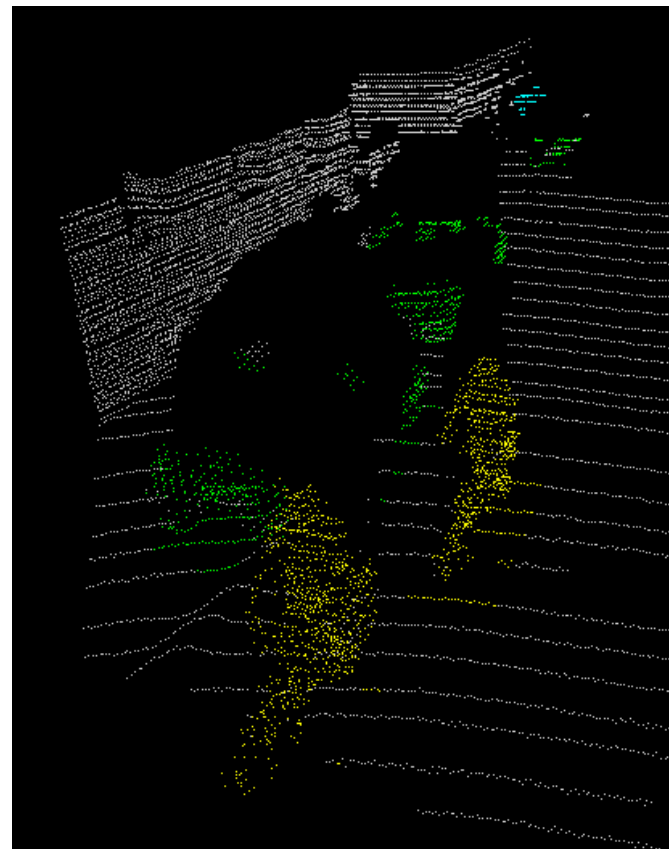
augmented point cloud

- 증강된 포인트 클라우드의 경우, 아래 사진을 보면 일정한 간격으로 찍힌 듯한 느낌이 있음
-> 복셀 기반으로 공간을 나누어 포인트가 있어야 한다고 판단할 경우 일정한 위치에 찍음
- 위 문제를 해결하기 위해 포인트가 밀집된 좌표로 예측 포인트를 이동시키도록 학습 (좌표 채널 추가)
- 추가로 바닥의 경우엔 RANSAC 알고리즘으로 바닥을 탐지하여 그 축에 해당하는 포인트들에 대해서는 포인트 증강 x

	Car	Pedestrian	Cyclist
GT	초록색	하늘색(cyan)	노란색
Pred	빨간색	보라색(magenta)	주황색



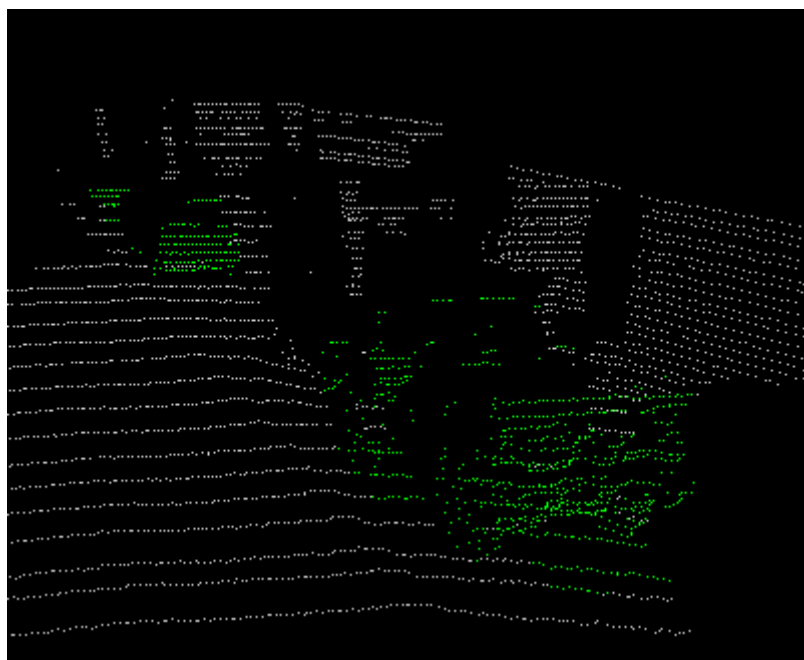
snow kitti point cloud



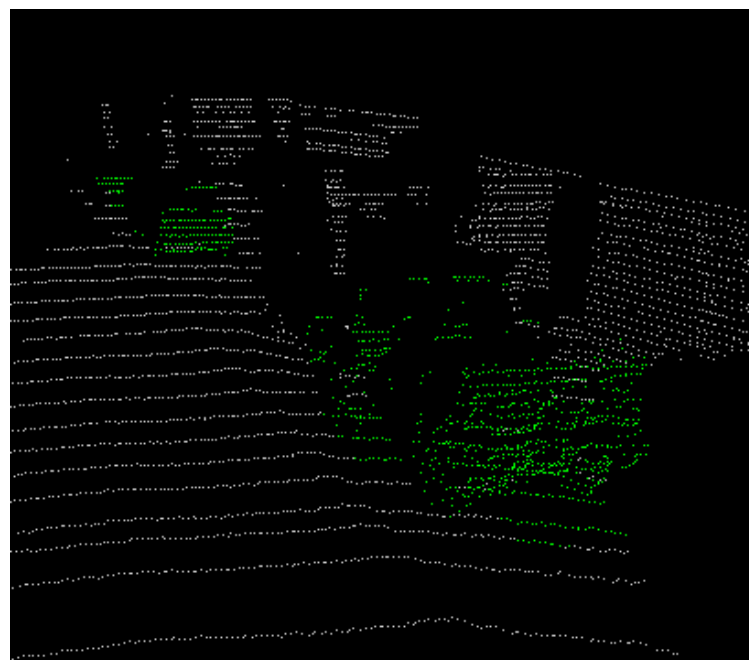
augmented point cloud

- 바닥의 경우엔 포인트가 증강되지 않는 것을 확인함. 또한 이전의 증강된 포인트의 간격이 덜 일정함.
- 하지만 evaluation 결과 확인 시 성능이 너무 안 좋은 것을 확인함.
-> gemini 활용하여 원인을 분석한 결과, 3DOD 모델들은 어떤 객체를 인식할 때, 반사율 등의 정보도 활용하기도 하며, 라이다 센서는 비슷한 위치의 포인트들에 대해 앞뒤로의 포인트를 생성하는 것이 아닌, 양옆으로 비슷한 위치의 포인트들을 생성하기에, 그 특정 패턴을 학습함.
-> 

	Car	Pedestrian	Cyclist
GT	초록색	하늘색(cyan)	노란색
Pred	빨간색	보라색(magenta)	주황색



snow kitti point cloud



augmented point cloud

- 밀도가 낮은 눈송이 노이즈를 일차적으로 제거함. 이전에 고정한 반사율을 학습할 수 있는 형태로 변경(채널 추가), 라이다 센서의 특징인 줄무늬 형태의 패턴 복원(증강된 포인트들을 정렬하는 방식), 가상의 라이다를 쏘아서 어떤 포인트 뒤의 겹치는 포인트가 있다면 성능을 떨어뜨릴 수 있으므로 제거
- 위 방식을 적용한 결과, 시각적으로도 포인트가 증강되는 것을 확인하였고, 실제 eval 결과도 상승하는 것을 확인하였음.
하지만 이번엔 포인트가 너무 제한적으로 찍혀서 성능 향상 폭이 제한적인 듯한 느낌을 받음
-> 추론 시 0.8 이상의 값을 만족하는 포인트만 남기는 것(엄격함)을 낮춤

Object-aware point cloud augmentation

Object-aware point cloud augmentation - evaluation results

point cloud augmentation _ PointPillars

model / method	Metric	Pedestrian AP@0.5	Cyclist AP@0.5	Car AP@0.7
PointPillars_kitti	3D-Bbox	51.4551 47.9575 43.8407	81.8821 63.6698 60.9022	86.6348 76.7492 74.1609
PointPillars_snow	3D-Bbox	50.9077 45.9942 42.0627	70.2602 44.4837 42.9534	55.0510 37.5991 32.3090
	BEV	56.2570 50.6632 47.2526	71.4513 46.5849 44.2080	74.9016 50.8983 49.5825
	2D-BBox	56.9435 51.8297 49.3069	76.5018 52.2800 49.3459	78.9322 59.3901 51.9992
	AOS	41.4645 38.1470 36.4075	75.5731 50.5951 47.5046	77.9229 58.2583 50.6567
object_aware_10_1	3D-Bbox	51.5069 47.4046 43.0504	68.6086 43.3760 41.8207	56.5922 38.9610 36.9650
	BEV	56.0670 51.0526 47.2730	70.3471 45.0353 43.0680	75.3290 51.2373 49.9452
	2D-Bbox	56.1920 52.0018 49.3976	75.9759 51.7205 48.7717	79.0481 59.5656 52.1319
	AOS	37.2625 35.5598 34.1988	74.6486 49.9139 46.8630	77.8570 58.3775 50.7919

point cloud augmentation _ VoxelRCNN

model / method	Metric	Pedestrian AP@0.5	Cyclist AP@0.5	Car AP@0.7
VoxelRCNN_kitti	3D-Bbox	66.5348, 60.2782, 55.6542	85.9466, 72.1112, 68.8626	89.6149, 83.8685, 78.8079
VoxelRCNN_snow	3D-Bbox	66.6909, 59.3986, 55.9992	83.5091, 59.1099, 55.1060	74.8540, 56.2723, 50.7544
	BEV	70.8167, 64.0834, 58.8447	87.0567, 60.0769, 57.7704	79.2096, 66.7585, 60.1988
	2D-BBox	75.7812, 68.8897, 66.4404	88.2552, 64.6207, 60.7499	89.7385, 71.0313, 70.0774
	AOS	70.80, 64.04, 61.58	87.88, 64.09, 60.24	89.37, 70.42, 69.21
object_aware_10_1	3D-Bbox	68.2522, 62.5165, 57.2864	83.2836, 57.8005, 54.8132	76.6414, 56.9756, 51.1320
	BEV	71.9412, 65.9402, 59.6342	86.6319, 59.3975, 57.6854	79.7229, 67.2637, 60.5713
	2D-Bbox	75.4026, 69.0739, 66.2658	87.7833, 63.9420, 60.0059	89.7881, 71.0415, 69.9019
	AOS	71.10, 64.51, 61.42	87.65, 63.60, 59.58	89.33, 70.39, 68.98

point cloud augmentation _ PointRCNN

model / method	Metric	Pedestrian AP@0.5	Cyclist AP@0.5	Car AP@0.7
PointRCNN_kitti	3D-Bbox	70.3937, 62.4997, 56.4812	86.8038, 67.5356, 64.1532	84.0957, 75.6573, 75.8359
PointRCNN_kitti_snow	3D-Bbox	69.2039, 61.6143, 55.8719	79.7640, 56.6199, 53.0094	64.7429, 52.0929, 48.6623
	BEV	73.4029, 65.4532, 59.0363	83.5402, 58.0971, 54.9253	77.3060, 64.4550, 59.9026
	2D-BBox	72.7548, 65.3306, 59.3042	83.7134, 58.8943, 56.7163	85.9689, 68.0673, 66.3882
	AOS	69.94, 62.63, 56.60	83.64, 58.65, 56.48	85.33, 67.30, 65.22
object_aware_10_1	3D-Bbox	69.8090, 62.3940, 54.8399	81.7320, 54.0137, 50.3843	64.7567, 49.0927, 48.4140
	BEV	72.9393, 65.4122, 58.5892	83.6675, 55.6279, 51.7060	77.0706, 63.5031, 59.3905
	2D-Bbox	69.4896, 64.1649, 58.2722	84.4226, 56.4499, 52.6338	85.7099, 67.6482, 65.6885
	AOS	66.59, 60.89, 55.02	84.37, 56.16, 52.41	85.03, 66.79, 64.51

point cloud augmentation _ mvmm

model / method	Metric	Pedestrian AP@0.5	Cyclist AP@0.5	Car AP@0.7
MVMM_kitti	3D-Bbox	58.0598, 54.6385, 51.5646	76.9789, 63.7341, 61.3158	88.7584, 78.1791, 77.2052
MVMM_kitti_snow	3D-Bbox	53.9292, 52.8448, 50.2185	71.5915, 50.4884, 47.6168	74.8244, 54.8129, 50.4703
	BEV	60.4902, 58.1564, 55.3361	78.2400, 55.0631, 52.3427	83.7933, 65.5323, 60.4207
	2D-BBox	66.9772, 64.8389, 62.6820	82.8715, 60.3086, 58.3068	87.8108, 69.2236, 67.2224
	AOS	62.7076, 60.2084, 57.7439	82.6868, 59.9001, 57.8186	87.5130, 68.5216, 66.3796
object_aware_10_1	3D-Bbox	55.8133, 53.3939, 50.2261	71.4356, 50.2268, 47.4286	74.2056, 54.5689, 50.2156
	BEV	60.7205, 58.0521, 55.3311	74.1496, 52.7853, 51.1222	83.1411, 65.2575, 60.2093
	2D-Bbox	66.8414, 64.2322, 62.1453	83.4482, 60.3835, 58.2898	87.4324, 68.8078, 66.9824
	AOS	61.4872, 58.8212, 56.4281	83.3039, 60.0295, 57.7904	87.0164, 68.0504, 66.0632

- Model_kitti : raw-kitti evaluation results
- Model_snow : snow-simulated kitti evaluation results
- Bold : better performance than snow-kitti evaluation

Object-aware point cloud augmentation - augmented point numbers

```
===== [ 000201 Scene GT Box Point Density Analysis ] =====
```

Obj ID	Class	Snow Pts	Pred Pts
01/12	Car	4	12
02/12	Car	731	955
03/12	Car	307	364
04/12	Cyclist	108	124
05/12	Car	104	127
06/12	Car	135	138
07/12	Car	62	62
08/12	Car	1	1
09/12	Car	8	8
10/12	Pedestrian	3	3
11/12	Pedestrian	2	2
12/12	Cyclist	67	70

```
=====
```

```
===== [ 001221 Scene GT Box Point Density Analysis ] =====
```

Obj ID	Class	Snow Pts	Pred Pts
01/11	Car	575	754
02/11	Cyclist	548	656
03/11	Car	197	214
04/11	Car	739	888
05/11	Car	102	102
06/11	Car	130	131
07/11	Car	26	28
08/11	Car	28	28
09/11	Pedestrian	8	8
10/11	Pedestrian	11	11
11/11	Cyclist	407	487

```
=====
```

```
===== [ 006024 Scene GT Box Point Density Analysis ] =====
```

Obj ID	Class	Snow Pts	Pred Pts
01/05	Car	419	540
02/05	Cyclist	32	32
03/05	Pedestrian	103	119
04/05	Pedestrian	187	245
05/05	Car	10	14

```
=====
```

```
===== [ 006496 Scene GT Box Point Density Analysis ] =====
```

Obj ID	Class	Snow Pts	Pred Pts
01/10	Car	412	530
02/10	Pedestrian	296	381
03/10	Pedestrian	118	127
04/10	Pedestrian	437	532
05/10	Cyclist	31	31
06/10	Pedestrian	148	150
07/10	Pedestrian	29	29
08/10	Cyclist	320	413
09/10	Car	1	1
10/10	Car	1	1

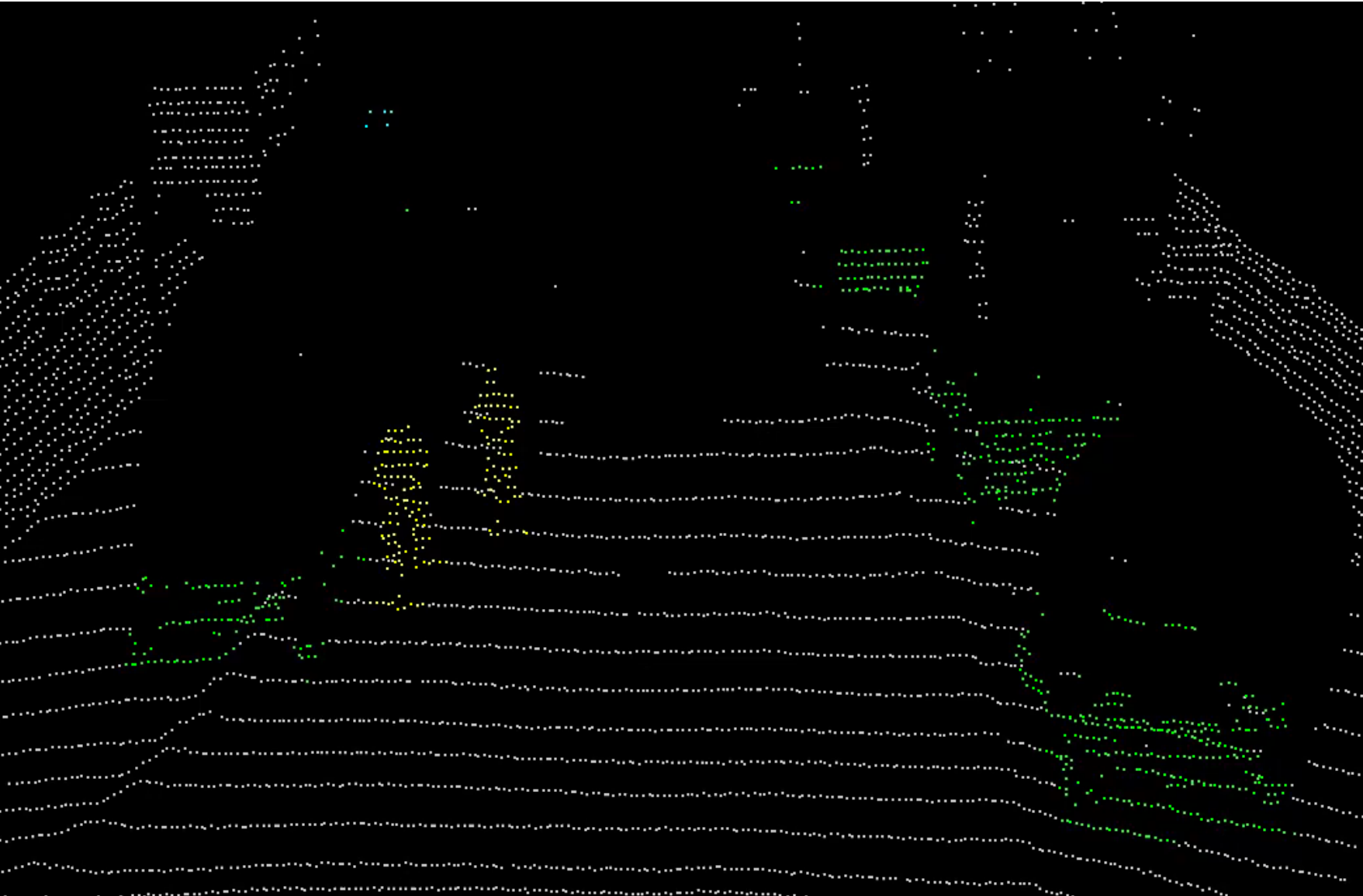
```
=====
```

```
===== [ 006813 Scene GT Box Point Density Analysis ] =====
```

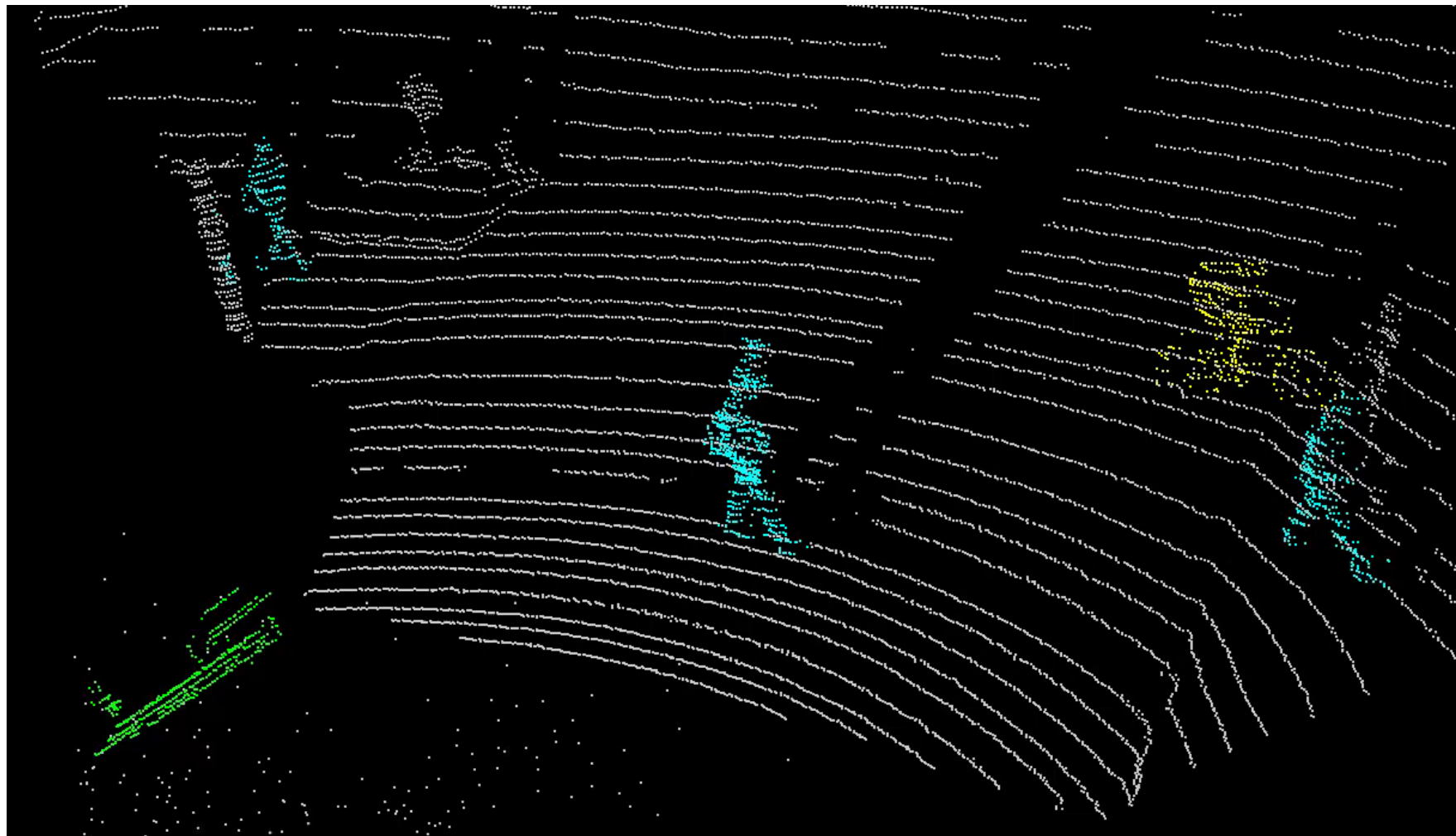
Obj ID	Class	Snow Pts	Pred Pts
01/13	Car	167	216
02/13	Cyclist	85	87
03/13	Cyclist	45	45
04/13	Pedestrian	45	49
05/13	Cyclist	12	13
06/13	Pedestrian	40	40
07/13	Pedestrian	22	22
08/13	Pedestrian	14	14
09/13	Cyclist	75	78
10/13	Pedestrian	18	18
11/13	Pedestrian	43	43
12/13	Pedestrian	21	21
13/13	Car	8	8

```
=====
```

- 그 이전의 방법들은 학습-추론 시간이 너무 오래 걸린다는 점이 한계라고 생각하였음. 포인트 노이즈를 제거하는 전처리 단계도 시간이 소요되어, 총 약 2시간의 처리 과정을 거쳐야 했음
- 3DOD에서는 실시간성이 중요하므로 이렇게 오래 걸리는 파이프라인은 경쟁력이 떨어진다고 생각하였음. 실제로 성능도 그렇게 좋지 않았음.
- 위의 한계를 개선하기 위해 노이즈 제거하는 전처리 프로세스를 없애고, 추론 시 불필요하다고 생각되는 부분들(RANSAC으로 바닥의 노이즈를 제거하는 등의 복잡하지만 비교적 불필요한 연산)을 제거하여 추론 시간을 기존 1시간 정도에서 3분으로 줄임
- 포인트가 많은 객체에 대해서는 증강되는 포인트를 분명히 확인할 수 있음
- 하지만 애초에 포인트가 적은 객체는 포인트의 특성을 네트워크가 파악할 수가 없으니 증강되는 포인트가 거의 없는 것을 확인할 수 있음



	Car	Pedestrian	Cyclist
GT	초록색	하늘색(cyan)	노란색
Pred	빨간색	보라색(magenta)	주황색



	Car	Pedestrian	Cyclist
GT	초록색	하늘색(cyan)	노란색
Pred	빨간색	보라색(magenta)	주황색

Writing paper
- abstract

Template for 2026 Summer Conference of the Korean Society of Broadcast
Engineers

*Sangwook Lee, *Chaeun Bae, *Donghun Lee, **Jinwoo Jeong, *Sang-hyo Park
*Kyungpook National University, **Korea Electronics Technology Institute

요약

3D 객체 탐지는 자율주행의 핵심 기술이나, 눈이나 비와 같은 악천후 환경에서는 센서 노이즈로 인해 심각한 성능 저하가 발생한다. 특히 실제 눈 내리는 환경의 3D 데이터셋 확보는 비용과 안전성의 문제로 매우 제한적이다. 본 논문에서는 이러한 데이터 부족 문제를 극복하고자, 기존 KITTI 데이터셋에 눈 시뮬레이션을 적용한 'SNOW-KITTI' 데이터셋을 구축하고, 모델들의 성능 저하 양상을 평가하였다. 구체적으로 3 종의 라이다 기반 탐지 모델과 1 종의 카메라-라이다 퓨전 모델을 선정하여 악천후 환경에서의 강건성을 비교 분석하였다. 흥미롭게도 복셀 기반의 라이다 모델은 구조적 노이즈 필터링 효과로 인해 높은 강건함을 보인 반면, 카메라-라이다 퓨전 모델은 다중 센서에 가해진 복합적인 노이즈로 인해 오히려 단일 모달리티보다 성능이 크게 하락하는 현상을 발견하였다. 본 연구는 눈 내리는 환경에서 3D 인지 모델의 구조적 취약성을 분석함으로써 향후 자율주행 시스템의 신뢰성을 높이기 위한 중요한 학술적 통찰을 제공한다.

- 초기 연구 주제는 '강설 환경에서의 3DOD 모델 성능 분석'이 주제였으나, 실험 진행하며 강설 환경에서의 3DOD 성능 향상을 위한 전처리 방식에 대해 고민하였습니다.
- 따라서 학습-추론의 과정을 거치는 전처리 프로세스를 구축하였으며, 자동차와 보행자 클래스에 대해서 성능이 일부 상승하는 것을 확인할 수 있었습니다.
- 현재 작성된 요약본은 이전의 연구 주제에 대해서 작성되었으며, 실험 결과를 어떤식으로 반영해서 작성하면 될지 여쭙고 싶습니다.
-> 3DOD를 위한 전처리 프레임워크 제안을 논지로 작성하면 될지?
- 실험 결과를 어떻게 반영할지에 따라 논문 제목도 ~~정해야하는 상황입니다.~~